复旦大学信息科学与工程学院

《计算机体系结构》期末考试试卷

2011～2012 学年第 2 学期

A 卷　共 12 页

课程代码：INFO130038.03　　考试形式：■开卷　□闭卷　　2012 年 6 月

（本试卷答卷时间为 120 分钟，答案必须写在试卷上，做在草稿纸上无效）

专业_____学号_____姓名_____成绩_____

题号	一	二	三	四	五	六	七	八	总分
得分									

一、(8 分)

考虑下面这个for循环程序的汇编代码：

```
<func>:
   push   %ebp
   mov    %esp,%ebp
   sub    $0x10,%esp
   movl   $0x0,-0x4(%ebp)
   movl   $0x0,-0x8(%ebp)
   jmp    L3
L1:
   mov    -0x8(%ebp),%eax
   shl    $0x2,%eax
   add    0x8(%ebp),%eax
   mov    (%eax),%eax
   cmp    -0x4(%ebp),%eax
   jle    L2
   mov    -0x8(%ebp),%eax
   shl    $0x2,%eax
   add    0x8(%ebp),%eax
   mov    (%eax),%eax
   mov    %eax,-0x4(%ebp)
L2:
   addl   $0x1,-0x8(%ebp)
L3:
   mov    -0x8(%ebp),%eax
   cmp    0xc(%ebp),%eax
```

```
    jl     L1
    mov    -0x4(%ebp),%eax
    add    %eax,%eax
    leave
    ret
```

根据以上的汇编代码，完成相应的for循环C程序代码。（请注意，在你的C代码中，你只能使用符号a，n和x来表示变量，而不能用寄存器名来表示）

```
int func(int a[5], int n)
{
int x;

_____________________________________________

_____________________________________________

_____________________________________________

_____________________________________________

_____________________________________________

_____________________________________________

_____________________________________________

return _____________________________________
}
```

二、(12 分)

在以下的问题中，你需要根据给出的结构（structure）和联合（union）的声明以及IA32 汇编代码，推断补全相应的C程序代码。

下面是要使用到的结构和联合的声明。

```
struct s1{          struct s2{              union u1{
    char c;             char b[4];              char d[7];
    int a[2];           struct s1 *p;           struct s2 *p2;
    };                  int num;                }
                        }
```

1. 下列各域在各自的结构或联合中的偏移量是多少（用字节表示）？

s1.a[1]: _____

s2.num : _____

u1.*p2 : _____

2. 对于下面左边的每个IA32 汇编代码，请在右边补全相应的C程序代码。（请注意下面的函数都需要返回值）

A: <proc1>:

```
push   %ebp
mov    %esp,%ebp
mov    0x8(%ebp),%eax
mov    0x8(%eax),%eax
pop    %ebp
ret
```

```
int proc1(struct s1* sv1)
{
_____________________________
_____________________________
_____________________________
}
```

B: <proc2>:

```
push   %ebp
mov    %esp,%ebp
sub    $0x10,%esp
movl   $0x0,-0x4(%ebp)
mov    0x8(%ebp),%eax
mov    0x8(%eax),%edx
mov    0x8(%ebp),%eax
mov    0x4(%eax),%eax
mov    0x4(%eax),%eax
lea    (%edx,%eax,1),%eax
mov    %eax,-0x4(%ebp)
mov    -0x4(%ebp),%eax
leave
ret
```

```
int proc2(struct s2* sv2)
{
_____________________________
_____________________________
_____________________________
_____________________________
_____________________________
}
```

C: <proc3>:

```
push   %ebp
mov    %esp,%ebp
sub    $0x10,%esp
mov    0x8(%ebp),%eax
mov    (%eax),%eax
mov    0x4(%eax),%eax
movzbl (%eax),%eax
mov    %al,-0x1(%ebp)
mov    0x8(%ebp),%eax
mov    (%eax),%eax
movzbl 0x2(%eax),%eax
and    %al,-0x1(%ebp)
movzbl -0x1(%ebp),%eax
leave
ret
```

```
int proc3(union u1* uv1)
{
_____________________________
_____________________________
_____________________________
_____________________________
_____________________________
}
```

三、(12 分)

你刚刚加入了一个编程小组，他们试图开发世界上最快的阶乘函数。开始时使用递归阶乘，他们将代码转换成使用迭代：

```
int fact(int n)
    {
        int i;
        int result=1;

        for(i=n; i>0; i--)
            result=result*i;
        return result;
    }
```

通过这样做，他们将函数的CPE数从63降低到4，这是在Intel Pentium III上测出的。不过，他们还想干得再好一点。

其中一个程序员听说过循环展开，她写出了如下代码

```
    int fact_u2(int n)
    {
        int i;
        int result=1;

        for(i=n; i>0; i-=3)
            result=（（result*i）*(i-1)）*(i-2);
        return result;
    }
```

不幸的是，小组发现这段代码对参数n的某些值返回为0.

A. 对于哪些n值，fact_u2和fact会返回不同的值？给出如何修正fact_u2。

B. 对fact_u2使用基准程序，显示性能没有修改。你如何解释这个现象呢？

C. 将循环的行为修改成

result=（result*（i*(i-1)））*(i-2);

你觉得性能会有改善吗？并解释为什么。

D. 对于result=（（result*i）*(i-1)）*(i-2)，我们可以用括号创建出计算的不同的结合，如result=（result*（i*(i-1)））*(i-2)。请给出结合的最佳形式，使得程序性能最好。

四、(12 分)

考虑下面的矩阵转置函数：

```
typedef    int array1[2][4]
typedef    int array2[4][2]
void transpose(array1 dst,    array2 src)
{
   int    i,    j;
      for (i=0;    i<4;    i++){
            for (j=0; j<2; j++){
                  dst[j][i]=src[i][j];
            }
      }
}
```

假设这段代码运行在一台如下属性的机器上“

- sizeof(int)=4。
- 数组src从地址0开始，而数组dst从地址32开始（十进制）。
- 只有一个L1数据高速缓存，它是直接映射、直写、写分配的，块大小为16字节。
- 这个高速缓存总共有32个数据字节，初始为空。
- 对src和dst数组的访问分别是惟一的读和写不命中的来源。

对于每个row和col，指明对src[row][col]和dst[row][col]的访问是命中（h）还是不命中（m）。例如读src[0][0]会不命中

src 数组				
	row 1	row 2	row 3	row 4
col 1				
col 2				

dst 数组				
	col 1	col 2	col 3	col 4
row 1				
row 2				

五、(12 分)

考虑如下一段程序：

```
#include "csapp.h"

int counter = 0;

void handler(int sig)
{
   counter += 2;
}

int count()
{
   signal(SIGCHLD, handler);
   int  i, status;
   for(i=0; i<2; i++)
   {
      counter++;
      if(fork() > 0)
      {
         while (waitpid(-1, &status, 0) > 0);
         return counter;
      }
   }
   return counter;
}

int main()
{
   int result = count();
   printf("counter =%d\n", result);
   exit(0);
}
```

问：

1. 程序运行一次，将输出几次 count 的值？请分析说明。

2. 每一次运行程序，是否总是有相同的输出？若是，请写出此固定输出；若否，请列出所有可能的输出。

六、(12 分)

请写出一段代码，利用 `fork()`函数创建一个子进程。子进程始终挂起，直到收到 `SIGINT` 信号后，打印出其父进程的 `PID`，再以状态 `2` 退出。父进程等待 `3` 秒后，向子进程发送 `SIGINT` 信号。接着，父进程显式地等待子进程结束并将其回收，并打印子进程的退出状态，最后以状态 `0` 退出。注意程序中用到的系统调用函数**不必**进行错误检查，用到的头文件已经给出。

```
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <signal.h>
#include <stdio.h>
```

七、(12 分)

下面这个问题考察你如何将虚拟地址翻译成物理地址。我们做出如下假设：

这个存储器是按字节寻址的；

存储器访问是针对 **1 字节的字**的（不是 4 字节的字）；

虚拟地址是 16 位长的；

物理地址是 13 位长的；

页面大小是 1024 字节；

TLB 是八路组相联，总共 16 个条目；

缓存是物理寻址、直接映射的，行大小为 4 字节，2 路相联总共 16 个组。

在下面的表格中，所有的数字均为 **16 进制**。

TLB 的内容、页表的前 32 页、缓存如下：

TLB			
Index	Tag	PPN	Valid
0	09	4	1
	12	2	1
	10	0	1
	03	5	1
	05	7	1
	13	1	0
	10	3	0
	18	3	0
1	04	1	0
	0C	1	0
	12	0	0
	08	1	0
	06	7	0
	03	1	0
	07	5	0
	02	2	0

Page Table					
VPN	PPN	Valid	VPN	PPN	Valid
00	6	1	10	0	1
01	5	0	11	5	0
02	3	1	12	2	1
03	4	1	13	4	0
04	2	0	14	6	0
05	7	1	15	2	0
06	1	0	16	4	0
07	3	0	17	6	0
08	5	1	18	1	1
09	4	0	19	2	0
0A	3	0	1A	5	0
0B	2	0	1B	7	0
0C	5	0	1C	6	0
0D	6	0	1D	2	0
0E	1	1	1E	3	0
0F	0	0	1F	1	0

2-way Set Associative Cache												
Index	Tag	Valid	Byte0	1	2	3	Tag	Valid	Byte0	1	2	3
0	19	1	99	11	23	11	00	0	99	11	23	11
1	15	0	4F	22	EC	11	2F	1	55	59	0B	41
2	1B	1	00	02	04	08	0B	1	01	03	05	07
3	06	0	84	06	B2	9C	12	0	84	06	B2	9C
4	07	0	43	6D	8F	09	05	0	43	6D	8F	09
5	0D	1	36	32	00	78	1E	1	A1	B2	C4	DE
6	11	0	A2	37	68	31	00	1	BB	77	33	00
7	16	1	11	C2	11	33	1E	1	00	C0	0F	00

1. 虚拟存储器的大小为_____KB，其中 PTE（页表条目）的数量共有_____个。

2. 对于虚拟地址 **0x41B5**，请你指出被访问的 TLB 条目，相对应的物理地址，以及返回的高速缓存字节值（用 16 进制表示）。并且请指出是否发生了 TLB 不命中，是否发生了缺页，是否发生缓存不命中。如果发生缓存不命中，在“返回的缓存字节”一栏中填“-”。如果发生缺页，在“PPN”栏中填入“-”并且空着 C 和 D 部分。

A. 填写虚拟地址（每格填一个 bit）

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

请说明下列符号的表示范围，例如[7-0]。

VPO （虚拟页面偏移）

VPN （虚拟页号）

B. 地址翻译

参数	值
VPN	0x
TLB索引	0x
TLB标记	0x
TLB命中？(Y/N)	
缺页？(Y/N)	
PPN	0x

C. 物理地址（每格填一个 bit）

12	11	10	9	8	7	6	5	4	3	2	1	0

请说明下列符号的表示范围，例如[7-0]。

CO （缓冲块内的字节偏移量）

CI （高速缓存索引）

CT （高速缓存标记）

D. 物理存储器引用

参数	值
字节偏移	0x
缓存索引	0x
缓存标记	0x
缓存命中？(Y/N)	
返回的缓存字节	0x